



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

ОТЧЕТ

по Лабораторной работе № 2

по дисциплине «**МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ**»

Тема: «**Обработка признаков (часть 2)**»

Студент	<u>ИУ5И-22М</u>	<u></u>	<u>Тавуз Мохамад</u>
	(группа)	(подпись, дата)	(И.О. Фамилия)
Преподаватель		<u></u>	<u>Ю.Е.Гапанюк</u>
		(подпись, дата)	(И.О. Фамилия)

2026 г.

1) Введение

Предварительная обработка данных является важным этапом при построении моделей машинного обучения. От качества подготовки признаков зависит корректность дальнейшего анализа данных и эффективность обучаемых моделей.

Цель данной лабораторной работы — изучение и применение продвинутых методов обработки признаков, включая масштабирование числовых признаков, обработку выбросов, формирование новых признаков на основе нестандартных данных, а также отбор наиболее значимых признаков различными методами.

В ходе работы был использован набор данных Titanic, который содержит информацию о пассажирах корабля: пол, возраст, класс билета, стоимость билета, порт посадки и другие признаки. В качестве целевой переменной используется признак `survived`, показывающий, выжил пассажир или нет.

В данной лабораторной работе были рассмотрены следующие методы:

- масштабирование числовых признаков с использованием `MinMaxScaler`, `StandardScaler` и `RobustScaler`;
- обработка выбросов методом удаления по межквартильному размаху `IQR`;
- замена выбросов методом ограничения значений `clipping`;
- обработка нестандартного текстового признака `embark_town`;
- отбор признаков методами `SelectKBest`, `RFE` и `RandomForestClassifier`.

О наборе данных Titanic:

В работе использован набор данных Titanic из библиотеки `seaborn`. Данный набор данных содержит информацию о пассажирах корабля Titanic и часто используется для демонстрации методов предварительной обработки данных и построения моделей классификации.

Основные характеристики набора данных:

- количество строк: 891;
- количество столбцов: 15;
- целевая переменная: survived;
- тип задачи: бинарная классификация.

Описание основных признаков:

- survived — целевая переменная, показывающая, выжил пассажир или нет;
- pclass — класс билета пассажира;
- sex — пол пассажира;
- age — возраст пассажира;
- sibsp — количество братьев, сестер или супругов на борту;
- parch — количество родителей или детей на борту;
- fare — стоимость билета;
- embarked — порт посадки;
- class — класс пассажира в текстовом виде;
- who — категория пассажира;
- adult_male — является ли пассажир взрослым мужчиной;
- deck — палуба;
- embark_town — город посадки;
- alive — текстовое представление целевой переменной;
- alone — путешествовал ли пассажир один.

2) Описание задания

В соответствии с заданием лабораторной работы необходимо выбрать один или несколько наборов данных и выполнить продвинутую предварительную обработку признаков.

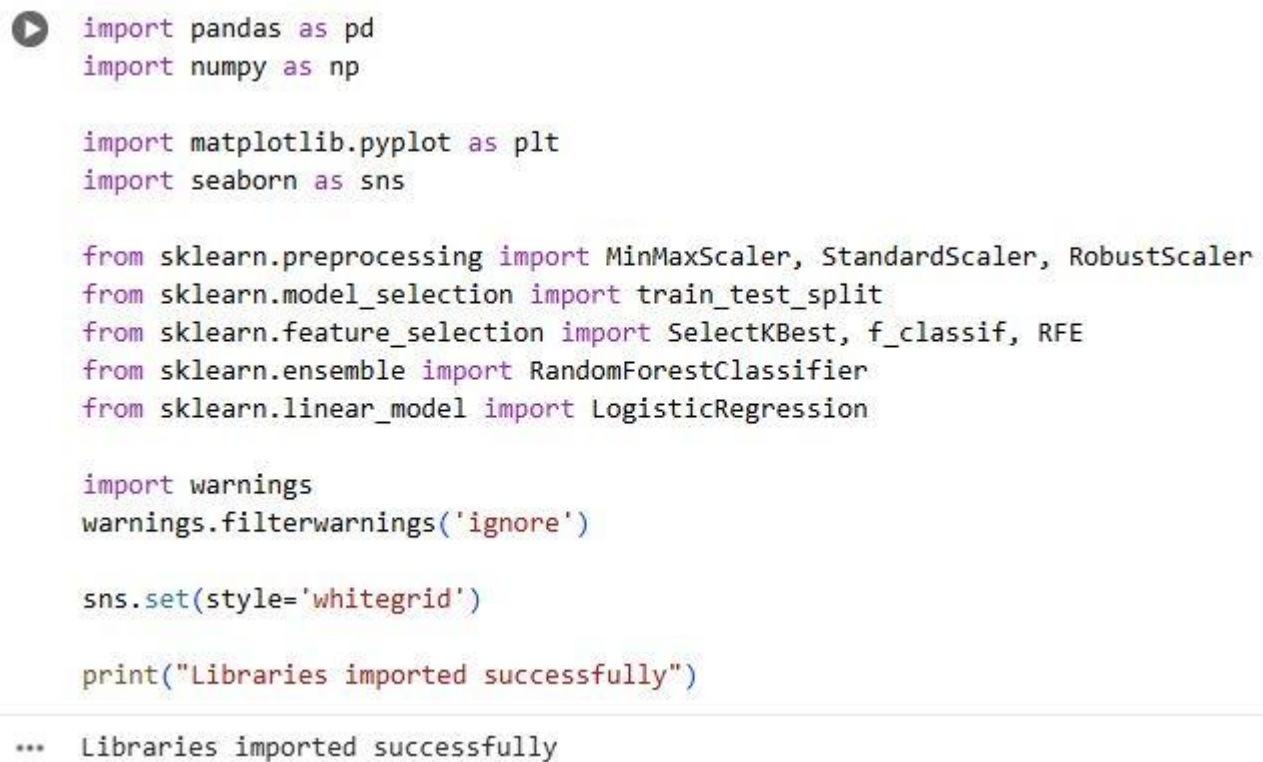
В данной работе использован набор данных Titanic. На его основе необходимо выполнить следующие этапы:

1. Загрузить набор данных и выполнить предварительный анализ структуры данных.
2. Проверить наличие пропущенных значений и выполнить их обработку.
3. Выполнить масштабирование числовых признаков не менее чем тремя способами:
 - MinMaxScaler;
 - StandardScaler;
 - RobustScaler.
4. Выполнить обработку выбросов для числового признака:
 - удалить выбросы методом межквартильного размаха IQR;
 - заменить выбросы методом ограничения значений clipping.
5. Выполнить обработку нестандартного признака, который не является обычным числовым признаком. В данной работе был использован текстовый признак `embark_town`.
6. Выполнить отбор признаков тремя методами:
 - метод фильтрации `SelectKBest`;
 - метод обертывания `RFE`;
 - метод вложений `RandomForestClassifier`.
7. Визуализировать результаты обработки данных с помощью графиков и сделать выводы.

3) Текст программы И Экранные формы с примерами выполнения

Импорт необходимых библиотек:

На первом этапе были подключены основные библиотеки, необходимые для загрузки, анализа, обработки и визуализации данных. Для работы с таблицами использовалась библиотека `pandas`, для численных вычислений — `numpy`. Для построения графиков были подключены `matplotlib` и `seaborn`. Также были импортированы инструменты библиотеки `scikit-learn`, необходимые для масштабирования признаков, отбора признаков и построения моделей машинного обучения.



```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import SelectKBest, f_classif, RFE
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression

import warnings
warnings.filterwarnings('ignore')

sns.set(style='whitegrid')

print("Libraries imported successfully")

... Libraries imported successfully
```

Рисунок 1 — Импорт необходимых библиотек

Загрузка набора данных и предварительный просмотр:

После подключения библиотек был загружен набор данных `Titanic` из библиотеки `seaborn`. Для первичного анализа были выведены первые пять строк таблицы. Это позволяет ознакомиться со структурой набора данных, названиями признаков и типом информации, содержащейся в столбцах.

```
# Загрузка набора данных Titanic
df = sns.load_dataset('titanic')

# Просмотр первых строк
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

Рисунок 2 — Загрузка набора данных Titanic и просмотр первых строк

Анализ структуры данных и проверка пропущенных значений:

На следующем этапе была выполнена проверка структуры набора данных с помощью метода `info()`. Данный метод позволяет определить количество строк, количество признаков, типы данных в столбцах и наличие пропущенных значений.

Также была выполнена отдельная проверка количества пропущенных значений в каждом столбце. В результате анализа было выявлено, что пропуски присутствуют в признаках `age`, `embarked`, `deck` и `embark_town`. Наибольшее количество пропусков содержится в признаке `deck`.

```
# Общая информация о наборе данных
df.info()

print("\n===== \n")

# Проверка количества пропущенных значений в каждом столбце
print(df.isnull().sum())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   survived              891 non-null    int64
1   pclass                891 non-null    int64
2   sex                   891 non-null    object
3   age                   714 non-null    float64
4   sibsp                 891 non-null    int64
5   parch                 891 non-null    int64
6   fare                  891 non-null    float64
7   embarked              889 non-null    object
8   class                 891 non-null    category
9   who                   891 non-null    object
10  adult_male            891 non-null    bool
11  deck                  203 non-null    category
12  embark_town           889 non-null    object
13  alive                 891 non-null    object
14  alone                 891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB

=====

survived      0
pclass        0
sex           0
age           177
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck          688
embark_town   2
alive         0
alone         0
dtype: int64

```

Рисунок 3 — Анализ структуры данных и проверка пропущенных значений

Обработка пропущенных значений:

После выявления пропущенных значений была выполнена их обработка. Для числового признака `age` пропущенные значения были заполнены медианным значением, так как медиана является устойчивой к выбросам характеристикой центральной тенденции.

Для категориальных признаков `embarked` и `embark_town` пропущенные значения были заполнены наиболее часто встречающимся значением, то есть модой. Признак `deck` был удален из набора данных, поскольку он содержал слишком большое количество пропущенных значений и мог отрицательно повлиять на дальнейшую обработку данных.

После выполнения обработки была проведена повторная проверка пропущенных значений, которая показала, что в подготовленном наборе данных пропуски отсутствуют.

```
# Создание копии исходного набора данных для дальнейшей обработки
df_processed = df.copy()

# Заполнение пропущенных значений в числовом признаке age медианой
df_processed['age'].fillna(df_processed['age'].median(), inplace=True)

# Заполнение пропущенных значений в категориальных признаках модой
df_processed['embarked'].fillna(df_processed['embarked'].mode()[0], inplace=True)
df_processed['embark_town'].fillna(df_processed['embark_town'].mode()[0], inplace=True)

# Удаление признака deck, так как он содержит слишком много пропущенных значений
df_processed.drop(columns=['deck'], inplace=True)

# Проверка результата обработки пропущенных значений
df_processed.isnull().sum()
```

	0
survived	0
pclass	0
sex	0
age	0
sibsp	0
parch	0
fare	0
embarked	0
class	0
who	0
adult_male	0
embark_town	0
alive	0
alone	0
dtype: int64	

Рисунок 4 — Обработка пропущенных значений

Масштабирование признаков методом Min-Max Scaling:

На данном этапе было выполнено масштабирование числовых признаков age и fare с использованием метода Min-Max Scaling. Данный метод преобразует значения признаков к диапазону от 0 до 1. Это позволяет привести признаки с различными единицами измерения к единому масштабу.

Для выполнения масштабирования был создан объект MinMaxScaler, после чего признаки age и fare были преобразованы и сохранены в отдельной копии набора данных.

```
# Выбор числовых признаков для масштабирования
numeric_cols = ['age', 'fare']

# Создание объекта MinMaxScaler
minmax_scaler = MinMaxScaler()

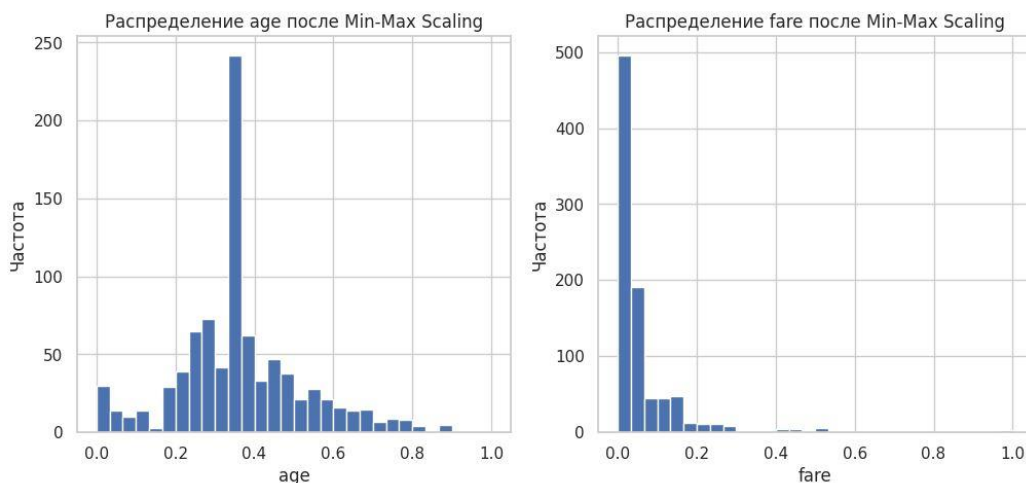
# Создание копии данных
df_minmax = df_processed.copy()

# Масштабирование признаков
df_minmax[numeric_cols] = minmax_scaler.fit_transform(df_minmax[numeric_cols])

# Просмотр результата
df_minmax[numeric_cols].head()
```

	age	fare
0	0.271174	0.014151
1	0.472229	0.139136
2	0.321438	0.015469
3	0.434531	0.103644
4	0.434531	0.015713

Рисунок 5 — Масштабирование признаков методом Min-Max Scaling



Масштабирование признаков методом StandardScaler:

На следующем этапе было выполнено масштабирование числовых признаков `age` и `fare` с использованием метода `StandardScaler`. Данный метод преобразует данные таким образом, что среднее значение признака становится равным 0, а стандартное отклонение — 1.

`StandardScaler` часто используется при подготовке данных для алгоритмов машинного обучения, чувствительных к масштабу признаков. В результате признаки `age` и `fare` были приведены к стандартному нормальному масштабу.

```
# Создание объекта StandardScaler
standard_scaler = StandardScaler()

# Создание копии данных
df_standard = df_processed.copy()

# Масштабирование признаков
df_standard[numeric_cols] = standard_scaler.fit_transform(df_standard[numeric_cols])

# Просмотр результата
df_standard[numeric_cols].head()
```

	age	fare
0	-0.565736	-0.502445
1	0.663861	0.786845
2	-0.258337	-0.488854
3	0.433312	0.420730
4	0.433312	-0.486337

Рисунок 7 — Масштабирование признаков методом `StandardScaler`

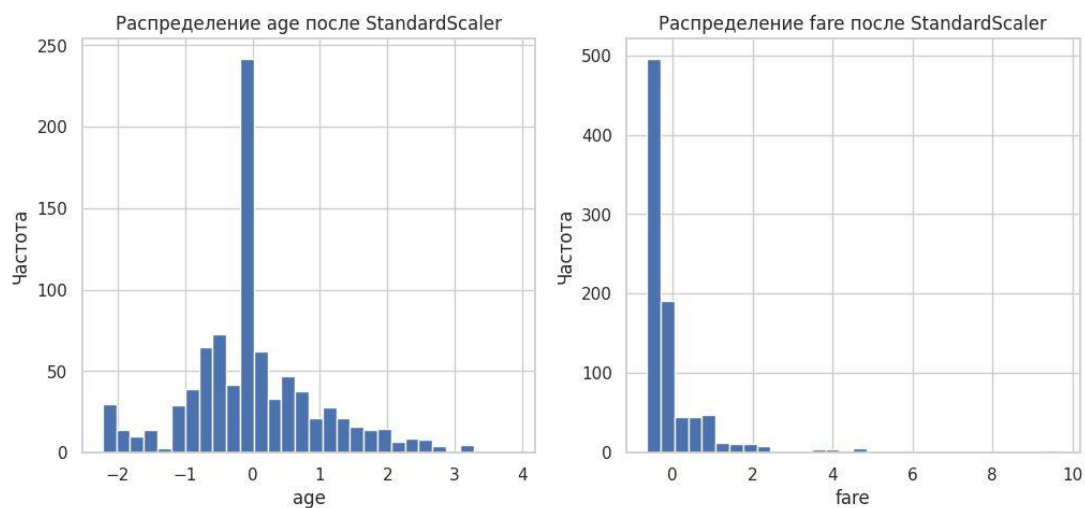


Рисунок 8 — Гистограммы признаков `age` и `fare` после `StandardScaler`

Масштабирование признаков методом RobustScaler:

Далее было выполнено масштабирование числовых признаков age и fare с использованием метода RobustScaler. В отличие от предыдущих методов, RobustScaler использует медиану и межквартильный размах, поэтому он является более устойчивым к выбросам.

Данный метод особенно полезен для набора данных Titanic, так как признак fare содержит заметные выбросы. После применения RobustScaler признаки age и fare были преобразованы с учетом устойчивых статистических характеристик.

```
# Создание объекта RobustScaler
robust_scaler = RobustScaler()

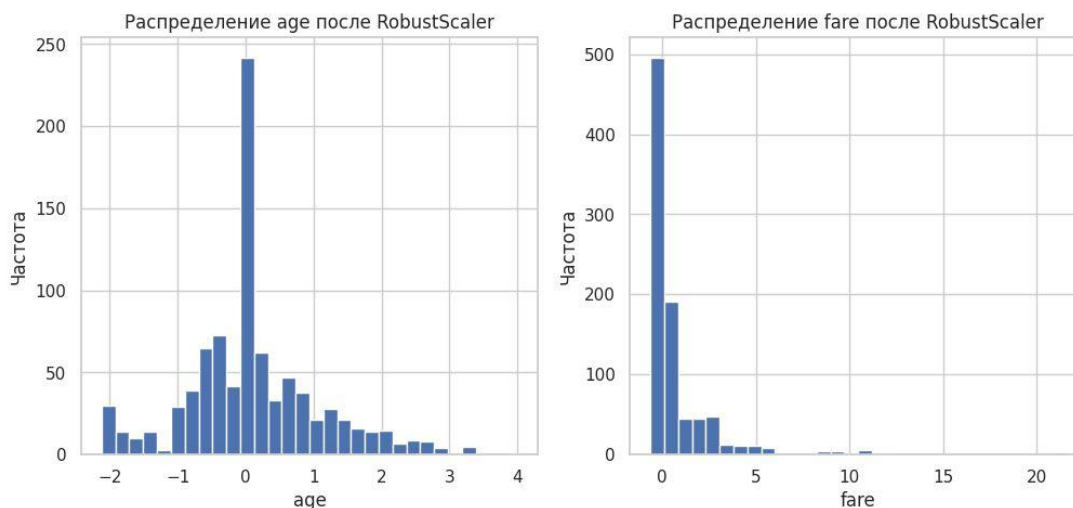
# Создание копии данных
df_robust = df_processed.copy()

# Масштабирование признаков
df_robust[numeric_cols] = robust_scaler.fit_transform(df_robust[numeric_cols])

# Просмотр результата
df_robust[numeric_cols].head()
```

	age	fare
0	-0.461538	-0.312011
1	0.769231	2.461242
2	-0.153846	-0.282777
3	0.538462	1.673732
4	0.538462	-0.277363

Рисунок 9 — Масштабирование признаков методом RobustScaler



Анализ выбросов в числовых признаках:

После выполнения масштабирования был проведен анализ выбросов в числовых признаках age и fare. Для визуального выявления выбросов были построены диаграммы размаха boxplot.

По результатам анализа видно, что признак fare содержит значительное количество выбросов, так как часть значений расположена далеко за пределами основного диапазона данных. Признак age также содержит отдельные нетипичные значения, однако наиболее выраженные выбросы наблюдаются именно в признаке fare. Поэтому дальнейшая обработка выбросов была выполнена для признака fare.

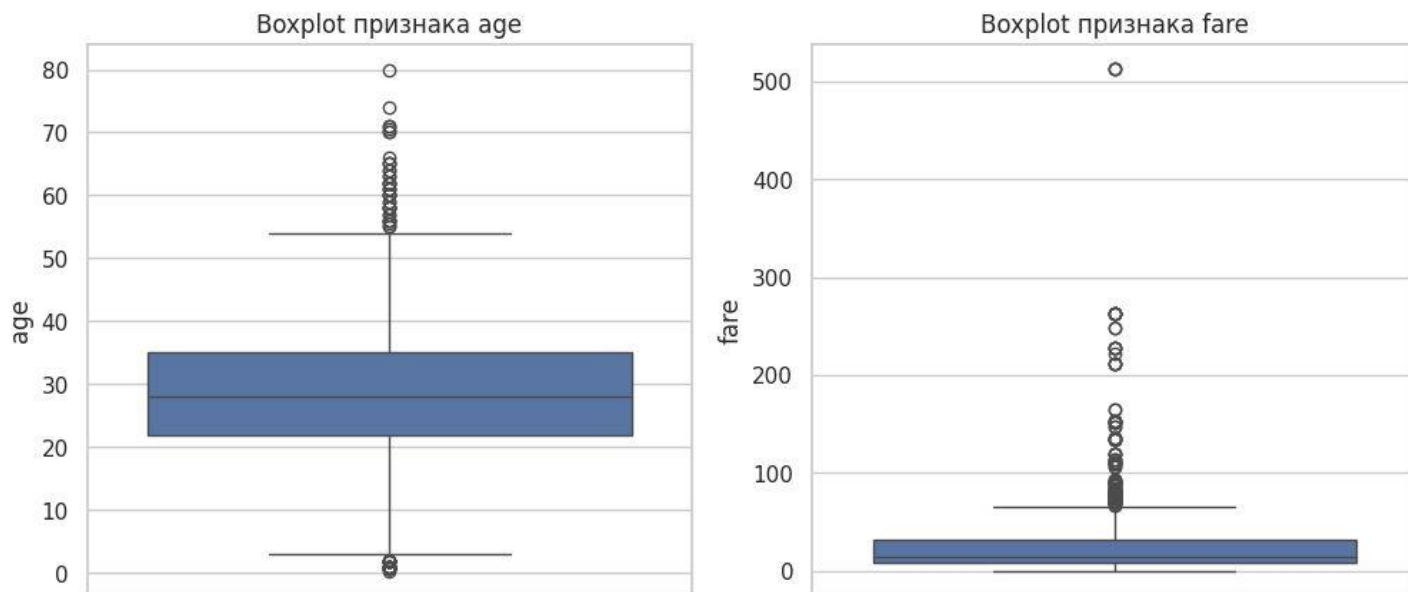


Рисунок 11 — Анализ выбросов в признаках age и fare

Удаление выбросов методом IQR:

Для удаления выбросов был использован метод межквартильного размаха IQR. Сначала для признака fare были вычислены первый квартиль Q1, третий квартиль Q3 и межквартильный размах IQR. Затем были определены нижняя и верхняя границы допустимых значений.

Все строки, в которых значение признака `fare` выходило за установленные границы, были удалены из набора данных. До удаления выбросов набор данных содержал 891 строку, после удаления — 775 строк. Таким образом, было удалено 116 строк, содержащих выбросы по признаку `fare`.

```
# Расчет квартилей и межквартильного размаха для признака fare
Q1 = df_processed['fare'].quantile(0.25)
Q3 = df_processed['fare'].quantile(0.75)
IQR = Q3 - Q1

# Определение нижней и верхней границы для выбросов
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Удаление строк, в которых fare выходит за допустимые границы
df_removed_outliers = df_processed[
    (df_processed['fare'] >= lower_bound) &
    (df_processed['fare'] <= upper_bound)
]

# Вывод количества строк до и после удаления выбросов
print("Количество строк до удаления выбросов:", df_processed.shape[0])
print("Количество строк после удаления выбросов:", df_removed_outliers.shape[0])
print("Количество удаленных строк:", df_processed.shape[0] - df_removed_outliers.shape[0])
print("Нижняя граница:", lower_bound)
print("Верхняя граница:", upper_bound)
```

Количество строк до удаления выбросов: 891
Количество строк после удаления выбросов: 775
Количество удаленных строк: 116
Нижняя граница: -26.724
Верхняя граница: 65.6344

Рисунок 12 — Удаление выбросов методом IQR

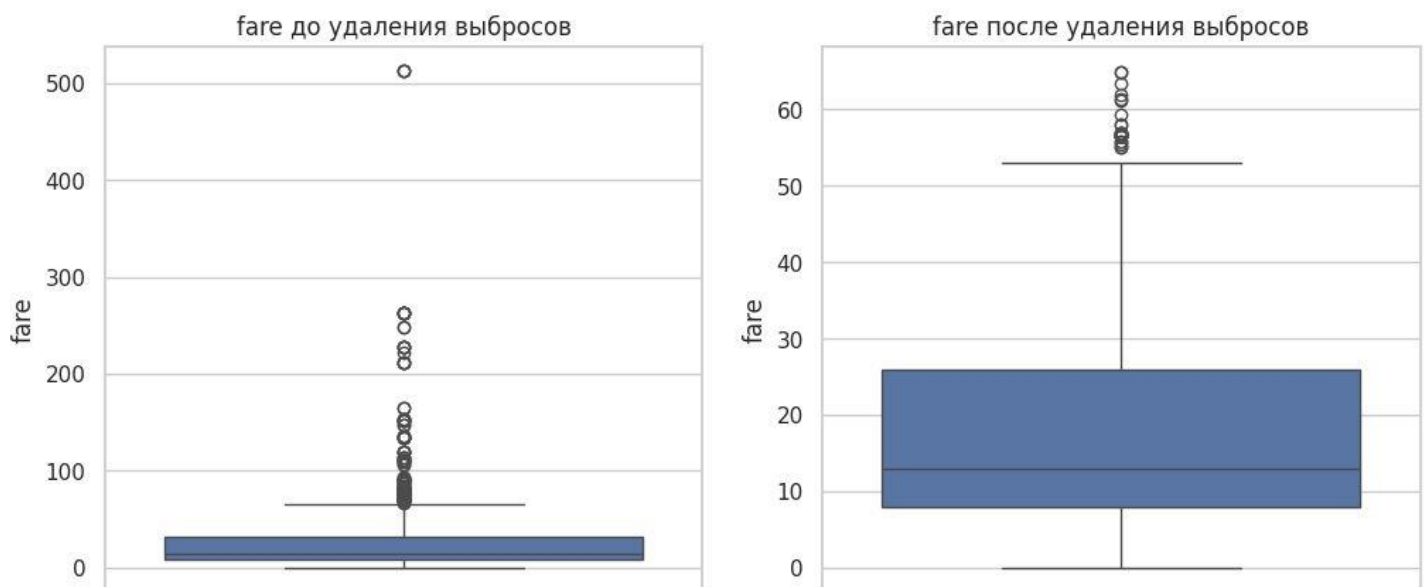


Рисунок 13 — Сравнение признака `fare` до и после удаления выбросов

Замена выбросов методом clipping:

Помимо удаления выбросов, была выполнена их замена методом ограничения значений clipping. В данном подходе строки из набора данных не удаляются. Вместо этого значения, выходящие за допустимые границы, заменяются на соответствующую нижнюю или верхнюю границу.

Для признака fare значения, превышающие верхнюю границу, были заменены на значение верхней границы. В результате максимальное значение fare уменьшилось с 512.3292 до 65.6344. Такой подход позволяет сохранить количество объектов в наборе данных и одновременно уменьшить влияние экстремальных значений.

```
# Создание копии данных для замены выбросов
df_replaced_outliers = df_processed.copy()

# Замена выбросов в признаке fare на нижнюю и верхнюю границы
df_replaced_outliers['fare'] = df_replaced_outliers['fare'].clip(
    lower=lower_bound,
    upper=upper_bound
)

# Вывод статистики до и после замены выбросов
print("Максимальное значение fare до замены:", df_processed['fare'].max())
print("Максимальное значение fare после замены:", df_replaced_outliers['fare'].max())

print("Минимальное значение fare до замены:", df_processed['fare'].min())
print("Минимальное значение fare после замены:", df_replaced_outliers['fare'].min())
```

Максимальное значение fare до замены: 512.3292
Максимальное значение fare после замены: 65.6344
Минимальное значение fare до замены: 0.0
Минимальное значение fare после замены: 0.0

Рисунок 14 — Замена выбросов методом clipping

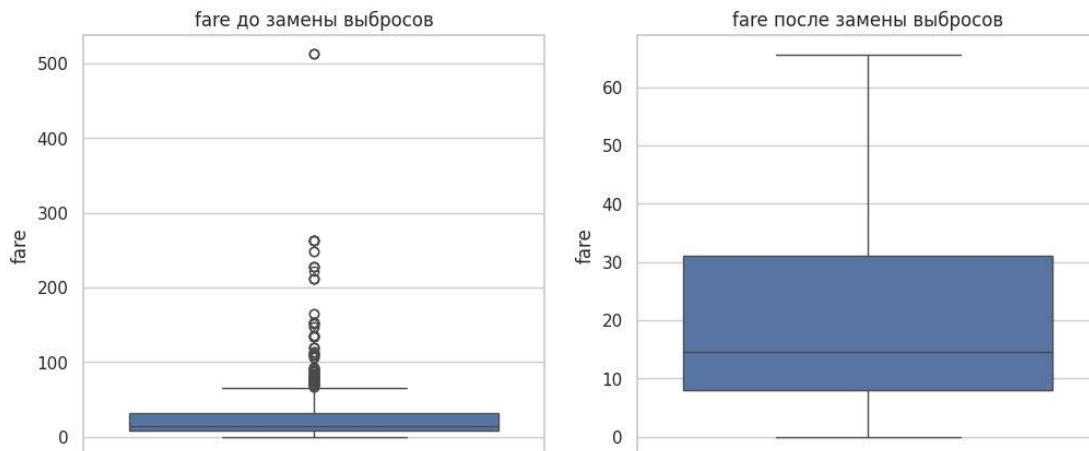


Рисунок 15 — Сравнение признака fare до и после замены выбросов

Обработка нестандартного текстового признака `embark_town`:

На следующем этапе была выполнена обработка нестандартного признака. В качестве такого признака был выбран текстовый признак `embark_town`, содержащий название города посадки пассажира.

На основе данного признака были сформированы два новых признака:

- `embark_town_length` — длина названия города посадки;
- `embark_town_first_letter` — первая буква названия города посадки.

Таким образом, текстовая информация была преобразована в новые признаки, которые могут быть использованы при дальнейшем анализе данных и построении моделей машинного обучения.

```
# Создание копии данных для обработки нестандартного признака
df_text_feature = df_replaced_outliers.copy()

# Формирование нового признака: длина названия города посадки
df_text_feature['embark_town_length'] = df_text_feature['embark_town'].astype(str).apply(len)

# Формирование нового признака: первая буква названия города посадки
df_text_feature['embark_town_first_letter'] = df_text_feature['embark_town'].astype(str).str[0]

# Просмотр результата
df_text_feature[['embark_town', 'embark_town_length', 'embark_town_first_letter']].head()
```

	embark_town	embark_town_length	embark_town_first_letter
0	Southampton	11	S
1	Cherbourg	9	C
2	Southampton	11	S
3	Southampton	11	S
4	Southampton	11	S

Рисунок 16 — Формирование новых признаков на основе `embark_town`

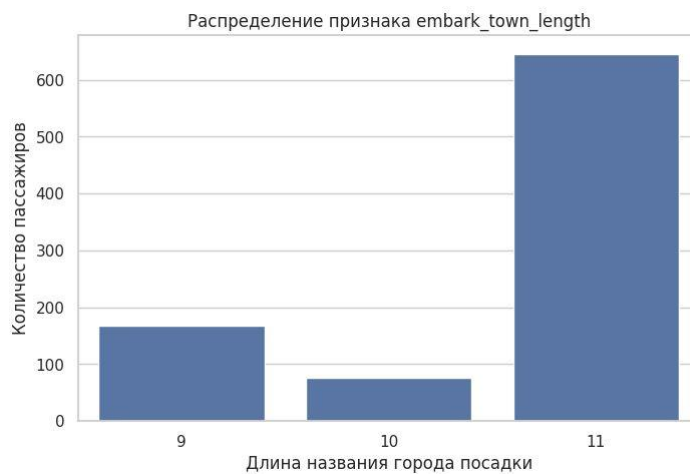


Рисунок 17 — Распределение нового признака `embark_town_length`

Подготовка данных для отбора признаков:

Перед применением методов отбора признаков была выполнена подготовка данных. Признак `alive` был удален, так как он напрямую дублирует целевую переменную `survived` и может привести к утечке данных.

Булевы признаки были преобразованы в числовой формат. Категориальные и текстовые признаки были закодированы методом One-Hot Encoding. После этого данные были разделены на матрицу признаков `X` и целевую переменную `y`.

В результате подготовки после кодирования было получено 19 признаков, которые далее использовались для применения методов отбора признаков.

```
# Создание копии данных для отбора признаков
df_features = df_text_feature.copy()

# Удаление признака alive, так как он напрямую дублирует целевую переменную survived
df_features.drop(columns=['alive'], inplace=True)

# Преобразование булевых признаков в числовой формат
bool_cols = df_features.select_dtypes(include=['bool']).columns
df_features[bool_cols] = df_features[bool_cols].astype(int)

# Кодирование категориальных и текстовых признаков методом One-Hot Encoding
df_features_encoded = pd.get_dummies(df_features, drop_first=True)

# Разделение данных на признаки X и целевую переменную y
X = df_features_encoded.drop(columns=['survived'])
y = df_features_encoded['survived']

# Просмотр размерности данных после кодирования
print("Размерность X:", X.shape)
print("Размерность y:", y.shape)

# Просмотр первых строк подготовленных данных
X.head()
```

Размерность X: (891, 19)
Размерность y: (891,)

	pclass	age	sibsp	parch	fare	adult_male	alone	embark_town_length	sex_male	embarked_Q	embarked_S	class_Second	class_Third	who_man	who_woman	embark_town_Queenstown	embark_town_Southampton	embark_town_f
0	3	22.0	1	0	7.2500	1	0	11	True	False	True	False	True	True	False	False	True	
1	1	38.0	1	0	65.6344	0	0	9	False	False	False	False	False	False	True	False	False	
2	3	26.0	0	0	7.9250	0	1	11	False	False	True	False	True	False	True	False	True	
3	1	35.0	1	0	53.1000	0	0	11	False	False	True	False	False	False	True	False	True	
4	3	35.0	0	0	8.0500	1	1	11	True	False	True	False	True	True	False	False	True	

Рисунок 18 — Подготовка данных для отбора признаков

Отбор признаков методом фильтрации SelectKBest:

На первом этапе отбора признаков был применен метод фильтрации SelectKBest. Методы фильтрации оценивают признаки независимо от конкретной модели машинного обучения и выбирают наиболее информативные признаки на основе статистического критерия.

В данной работе использовалась функция `f_classif`, которая оценивает связь каждого признака с целевой переменной `survived`. Было выбрано 5 наиболее значимых признаков.

В результате применения SelectKBest были выбраны следующие признаки:

- `pclass`;
- `adult_male`;
- `sex_male`;
- `who_man`;
- `who_woman`.

```
# Преобразование всех признаков в числовой формат
X_numeric = X.astype(int)

# Создание объекта SelectKBest для выбора 5 лучших признаков
filter_selector = SelectKBest(score_func=f_classif, k=5)

# Обучение метода фильтрации
X_filter_selected = filter_selector.fit_transform(X_numeric, y)

# Получение выбранных признаков
filter_selected_features = X_numeric.columns[filter_selector.get_support()]

# Вывод выбранных признаков
print("Выбранные признаки методом SelectKBest:")
print(list(filter_selected_features))
```

```
Выбранные признаки методом SelectKBest:
['pclass', 'adult_male', 'sex_male', 'who_man', 'who_woman']
```

Рисунок 19 — Отбор признаков методом SelectKBest

Отбор признаков методом обертывания RFE:

На следующем этапе был применен метод обертывания RFE, то есть Recursive Feature Elimination. В отличие от методов фильтрации, методы обертывания используют модель машинного обучения для оценки качества признаков.

В данной работе в качестве базовой модели для RFE была использована логистическая регрессия. Метод последовательно исключал менее значимые признаки и оставил 5 наиболее важных признаков для предсказания целевой переменной survived.

В результате применения RFE были выбраны следующие признаки:

- pclass;
- sibsp;
- adult_male;
- alone;
- who_man.

```
# Создание модели логистической регрессии для метода RFE
rfe_model = LogisticRegression(max_iter=1000)

# Создание объекта RFE для выбора 5 лучших признаков
rfe_selector = RFE(estimator=rfe_model, n_features_to_select=5)

# Обучение метода RFE
rfe_selector.fit(X_numeric, y)

# Получение выбранных признаков
rfe_selected_features = X_numeric.columns[rfe_selector.support_]

# Вывод выбранных признаков
print("Выбранные признаки методом RFE:")
print(list(rfe_selected_features))
```

Выбранные признаки методом RFE:
['pclass', 'sibsp', 'adult_male', 'alone', 'who_man']

Рисунок 20 — Отбор признаков методом RFE

Отбор признаков методом вложений RandomForestClassifier:

На заключительном этапе был применен метод вложений на основе модели RandomForestClassifier. Методы вложений выполняют отбор признаков в процессе обучения модели и позволяют определить важность каждого признака.

После обучения модели RandomForestClassifier были рассчитаны значения важности признаков. Затем признаки были отсортированы по убыванию важности, и были выбраны 5 наиболее значимых признаков.

В результате применения RandomForestClassifier были выбраны следующие признаки:

- age;
- fare;
- adult_male;
- who_man;
- sex_male.

```
# Создание модели RandomForestClassifier
rf_model = RandomForestClassifier(random_state=42)

# Обучение модели
rf_model.fit(X_numeric, y)

# Получение важности признаков
feature_importances = pd.DataFrame({
    'Признак': X_numeric.columns,
    'Важность': rf_model.feature_importances_
})

# Сортировка признаков по важности
feature_importances = feature_importances.sort_values(
    by='Важность',
    ascending=False
)

# Выбор 5 наиболее важных признаков
embedded_selected_features = feature_importances.head(5)

# Вывод результата
print("Выбранные признаки методом RandomForestClassifier:")
print(list(embedded_selected_features['Признак']))

embedded_selected_features
```

Выбранные признаки методом RandomForestClassifier:
['age', 'fare', 'adult_male', 'who_man', 'sex_male']

	Признак	Важность
1	age	0.259963
4	fare	0.168553
5	adult_male	0.109671
13	who_man	0.092559
8	sex_male	0.087445

Рисунок 21 — Отбор признаков методом RandomForestClassifier

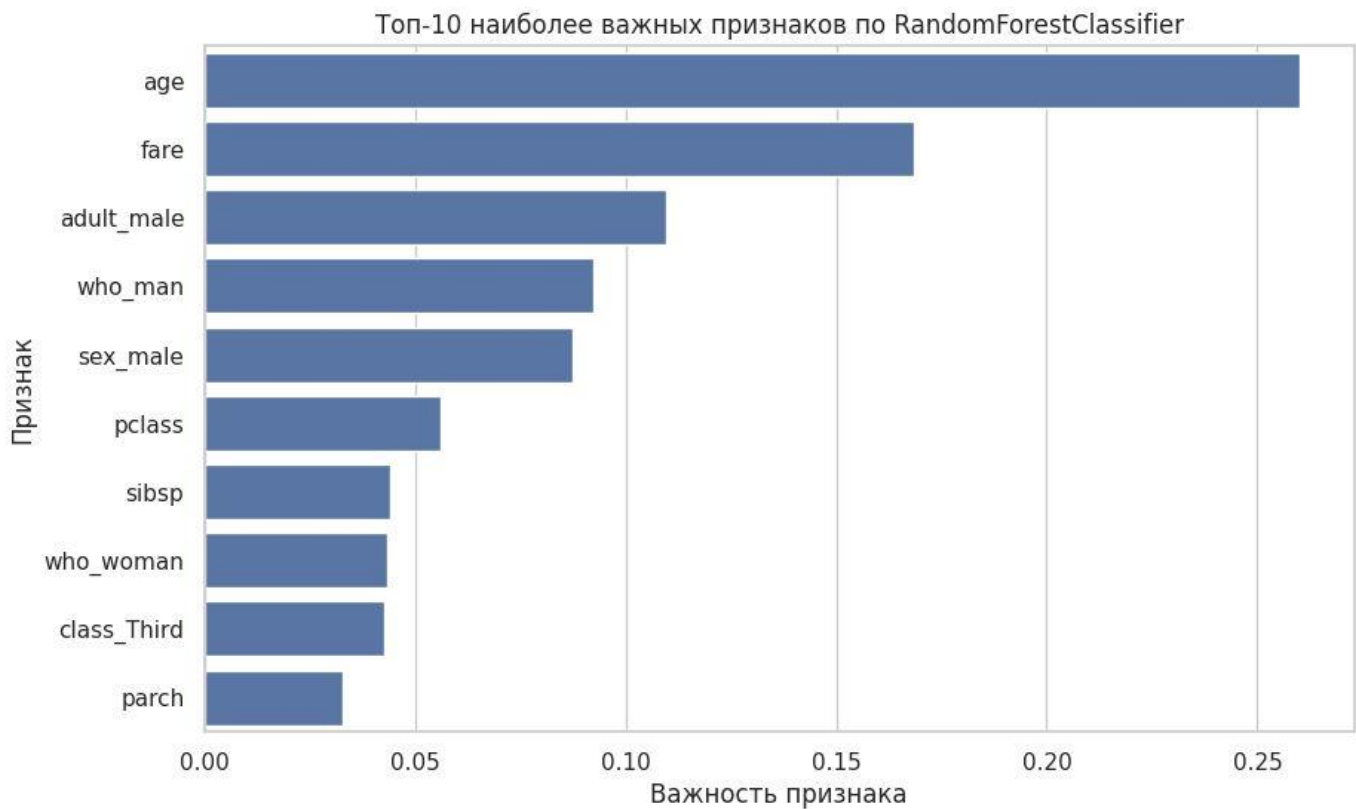


Рисунок 22 — Визуализация важности признаков RandomForestClassifier

4) Заключение

В ходе лабораторной работы были изучены и применены продвинутые методы предварительной обработки признаков на наборе данных Titanic.

Были выполнены следующие этапы работы:

1. Загружен и проанализирован набор данных Titanic.
2. Выполнена проверка и обработка пропущенных значений.
3. Выполнено масштабирование числовых признаков age и fare тремя способами:
 - MinMaxScaler;
 - StandardScaler;
 - RobustScaler.
4. Выполнена обработка выбросов в признаке fare:

- удаление выбросов методом межквартильного размаха IQR;
- замена выбросов методом clipping.

5. Выполнена обработка нестандартного текстового признака `embark_town`, на основе которого были сформированы новые признаки `embark_town_length` и `embark_town_first_letter`.

6. Выполнен отбор признаков тремя группами методов:

- SelectKBest как метод фильтрации;
- RFE как метод обертывания;
- RandomForestClassifier как метод вложений.

В результате работы было показано, что разные методы предварительной обработки данных позволяют улучшить качество признакового пространства и подготовить данные для дальнейшего построения моделей машинного обучения. Наиболее значимыми признаками для предсказания выживаемости пассажиров оказались `age`, `fare`, `adult_male`, `who_man`, `sex_male` и `pclass`.